

Jackson Holt

Dr. Moorman

Operating Systems

Transylvania University

Elves, Orcs, and Dwarfs

INTRODUCTION

Threads allow for computer to execute processes concurrently. As we are aware, this causes several synchronization problems. What thread is allowed to access what resource at what time? What happens when a thread is preempted and it is halfway done manipulating a shared piece of data? What happens if one thread is trying to read something that is halfway done being modified? This section of the code that is accessing the shared data is known as the critical section. If we want to ensure synchronization in the critical section, we must meet three criteria: mutual exclusion, bounded wait, and progress. As we move through the analysis of synchronization principles and the project, we will see how we are able to implement these principles with the bathroom example.

It is important to note that when I was implementing the solution to the bathroom problem I was only able to get it working for one bathroom. Therefore, in this paper, we will discuss the problem assuming one bathroom, then discuss the implementation of multiple bathrooms afterward.

THE BATHROOM PROBLEM

The goal of my program was to solve a multi-species bathroom problem. In this fantastic fantasy world, there are three species: elves, Dwarfs, and Orcs. All bathrooms are free to be used by anyone. If a student needs to go to the bathroom, they can enter if it is empty or only occupied by members of their species. Otherwise, they must wait. If a member of a different species is waiting, then folks in the bathroom can only exit.

We need to guarantee mutual exclusion when one species is in the bathroom, another cannot enter or access the bathroom. We need to guarantee bounded wait by only letting things leave if there is another species waiting. And we must have progress by only letting the current species in the bathroom, and let another one enter after all of the original species have left the bathroom.

MUTUAL EXCLUSION

Mutual exclusion is the idea that if process P_i is operating in its critical section, then no other process can be executing in its critical section. We can guarantee this using a type of locking mechanism. In the general case, we want to lock other processes out of entering their critical section. Unfortunately these create busy wait, but it is the easiest way for us to implement mutual exclusion.

For implementing bounded one of these we will be using semaphores. We are able to post wait and post semaphores in order to gain and return access to the process's critical section. The wait function with semaphores increments the counter and potentially unblocks any waiting processes. We implemented 4 semaphores, not including the printing semaphore, to create the solution. The first set of semaphores is one for each of the species. Since multiple threads representing one of each species will be trying to enter and exit the bathroom at the same time so each individual elf, orc, or dwarf is only modifying relevant information about their respective species one at a time. The last semaphore is the bathroom semaphore, which allows for one species to have access to the bathroom. Only after all of one species has left the bathroom will the bathroom semaphore decrement (post) and be able to let another species thread have access.

These semaphores allow for mutual exclusion by only allowing modification to the shared data of the bathroom to the species that is in the bathroom, and then only allowing shared species information to be modified by one individual member of that species.

BOUNDED WAIT

Bounded wait is the idea that when a thread has requested access to the critical section, it will be given access at some point and not starve and wait forever. In our example, we want to make sure that one species is not being locked out of the bathroom by another.

For bounded wait, we needed a way of keeping track of whether there are other waiting species. If there is a waiting species, all of the current species should exit one by one and let the waiting species into the restroom. This is best described using an example. If we have elves in the bathroom currently, and an orc would like to enter the restroom, then once the waiting orc counter is equal, it waits on the bathroom semaphore requesting to enter the bathroom as the first of its species. As soon as all of the elves have left and there are no more waiting elves, the bathroom semaphore is posted, allowing the orcs to then enter and use the bathroom freely. This means that no species will ever be starved and not be let in.

PROGRESSION

Progression is the idea that if no thread is in a critical section then if there is something waiting to enter its critical section, it should be allowed to enter. For the bathrooms this means that if there is nothing in the bathroom and a fantasy creature wishes to enter the bathroom then we should not keep them waiting outside of the bathroom.

To accomplish this we have our waiting species counter. If there is a waiting species and the first thread of that species will be able to request the bathroom semaphore, and if it is open, then the fantasy creature will be let into the bathroom at that point. meaning there is no waiting if nothing is using the bathroom semaphore, then that species will be let into the restroom.

1 MULTIPLE BATHROOMS

Since I was unable to implement more than one bathroom I will use this section to discuss my intended implementation. For each of the bathrooms you would create an array of bathroom

semaphores, each representing one bathroom. By requesting the semaphore of the bathroom and not getting access to it then you should move on to another bathroom. However I could not figure out a mechanism to easily accomplish the switching. Since you cannot easily track which bathrooms are full and which are open, and posting and waiting on semaphores like that created a knot I was not able to undo. I think there would be a better solution using a counting semaphore since those are used to access pools of resources, which would be each of the bathrooms.

CONCLUSION

The bathroom problem serves as a strong demonstration of how synchronization principles can be applied to real-world multi-threaded systems. Through the use of semaphores and careful tracking of waiting and active species, we are able to enforce the three critical requirements of synchronization: mutual exclusion, bounded wait, and progress. Mutual exclusion ensures that no two species use the bathroom at the same time, bounded wait guarantees that no group is indefinitely delayed from entering, and progress ensures that waiting threads are not ignored when the bathroom becomes available.

Although the implementation presented here successfully models a single bathroom, extending the solution to multiple bathrooms presents additional challenges, particularly in managing shared resources and ensuring fairness across all bathrooms. Nonetheless, the problem provided valuable insights into thread coordination, resource allocation, and the practical difficulties of avoiding deadlock and starvation. By approaching the synchronization problem through this creative lens of elves, orcs, and dwarves, we gain a deeper appreciation for the complexity and elegance of concurrent programming solutions.